

GEMINI API PARAMETERS ASSIGNMENT REPORT

1. Executive Parameter Summary

maxOutputTokens: Controls the maximum length of the model's generated response (measured in tokens). **Practical Scenario:** This is tuned to low values (e.g., 50–100) in search-snippet generation or mobile UI panels to prevent long text from breaking layouts, and tuned high (e.g., 8192) for complete code refactoring tasks.

temperature: Controls the randomness of token selection. Lower values (near 0.0) lead to highly deterministic, repeatable, and analytical outputs. Higher values (near 1.0) encourage creativity, linguistic diversity, and risk-taking. **Practical Scenario:** Tuned to 0.0 for structured JSON parsing and SQL-query translation where consistency is paramount, and tuned to 0.9 for creative brainstorming assistants or game dialogue generators.

stopSequences: Specifies set strings which immediately halt text generation if produced by the model. **Practical Scenario:** Used in few-shot prompting to specify ["\n", "User: "] to prevent the model from generating both sides of a conversation or continuing to mock dialogue.

stream (generateContentStream): Delivers response chunks progressively as they are computed rather than waiting for the complete response. **Practical Scenario:** Highly critical for user-facing chatbots (e.g. customer support) to reduce perceived latency and maintain high user engagement by showing immediate text progression.

responseLogprobs & logprobs: Returns log probability scores for chosen tokens along with the top N alternate candidate tokens. **Practical Scenario:** Tuned in automated transcription or medical information extraction pipelines to measure model confidence and automatically route low-probability extractions to a human auditor.

The Logit Bias Gap

The "logit bias gap" refers to the architectural difference where OpenAI's API allows developers to pass a `logit_bias` dictionary to directly modify logit probabilities (forcing or banning specific tokens), whereas the Gemini API has no native equivalent. While Gemini supports read-only logprobs to observe probability, developers cannot programmatically influence them at the token level during decoding. To bridge this gap, Gemini developers must rely on alternative approaches such as strict few-shot examples, system instructions (e.g. "Do not use word X"), schema enforcement, or post-generation validation and regeneration. This represents a gap in strict token-level determinism and absolute vocabulary control.

2. Captured Experiment Log Output

The following log records the execution of the parameters script. Note the premature termination due to `MAX_TOKENS` in Experiment 1, the minor variations in Experiment 2 despite `temperature = 0.0` (a common GPU concurrency non-determinism), the early termination in Experiment 3 due to the stop sequence, the chunked delivery in Experiment 4, and the expected API client error demonstrating logprobs limitations on Flash models in Experiment 5.

```
=====
GEMINI API PARAMETERS EXPERIMENTS LOG
Model used: gemini-2.5-flash
=====

[EXPERIMENT 1] Parameter: maxOutputTokens
Testing max_output_tokens = 10 (should truncate response prematurely)
Prompt: 'Explain how photosynthesis works in detail.'
Output (truncated):
None
Finish Reason: FinishReason.MAX_TOKENS
-----

[EXPERIMENT 2] Parameter: temperature
Prompt: 'Write a one-sentence description of a futuristic city.'

--- Low Temperature (0.0) Run 3 times (expect identical/deterministic outputs) ---
Run 1: Beneath a canopy of shifting holographic skies, the hyper-connected metropolis rises in gleaming, self-sustaining spires interwoven with luminous transit ribbons.
Run 2: Beneath a canopy of shifting holographic skies, the hyper-connected metropolis rises in gleaming, self-sustaining spires interwoven with luminous transit ribbons.
Run 3: Beneath a canopy of shifting holographic skies, the vertical metropolis hums with silent energy as gravity-defying vehicles weave between crystalline towers.

--- High Temperature (1.0) Run 3 times (expect diverse/creative outputs) ---
Run 1: Towering spires of bio-engineered glass and terraced gardens pierce the clouds of this vertical metropolis, where silent, glowing transit craft weave across the skyline.
Run 2: Beneath a web of glowing holographic skyways, towering crystalline spires wrapped in lush vertical gardens rise into a sky bustling with autonomous drones.
Run 3: The vertiginous metropolis pulsed with life, its gleaming glass spires and lush vertical gardens bound together by a web of silent, magnetic-levitation highways.
```

```

-----
[EXPERIMENT 3] Parameter: stopSequences
Testing stop_sequences=['process', 'chlorophyll'] (should stop when these words are generated)
Prompt: 'Explain photosynthesis in a simple way.'
Output (stopped early): Imagine if you could make your own dinner just by standing outside in the sunshine. That is
basically what plants do! **Photosynthesis** is the
Finish Reason: FinishReason.STOP
-----

```

```

[EXPERIMENT 4] Parameter: stream (generateContentStream)
Streaming output chunk-by-chunk:
Prompt: 'Write a short poem about the ocean.'
Output chunks: [A giant cradle, deep and wide,][
Swayed by the pulling of the tide,][
It whispers secrets to the shore,][
An ancient song forevermore.][

In shades of sapphire, green, and foam,][
A wild and everlasting home,][
Where quiet realms of shadow sleep,][
Beneath the wonders of the deep.] []
-----

```

```

[EXPERIMENT 5] Parameters: responseLogprobs & logprobs
Attempting to query logprobs on gemini-2.5-flash...
Expected Error Caught!
Error Message: 400 INVALID_ARGUMENT. {'error': {'code': 400, 'message': 'Logprobs is not enabled for this model',
'status': 'INVALID_ARGUMENT'}}
Explanation: Gemini Flash models do not support the logprobs feature. Pro models support it, but are restricted in
standard free quotas.
=====

```

3. Python Source Code (gemini_experiments.py)

Below is the complete Python script used to execute the experiments, clearly organized and documented per parameter. It handles rate limiting to prevent 429 errors on the Gemini API free tier.

```

import os
import sys
import time
from google import genai
from google.genai import types

def run_experiments():
    # Verify API key
    if "GEMINI_API_KEY" not in os.environ:
        print("Error: GEMINI_API_KEY environment variable is not set.")
        sys.exit(1)

    client = genai.Client()
    model_name = "gemini-2.5-flash"

    print("=" * 60)
    print("GEMINI API PARAMETERS EXPERIMENTS LOG")
    print(f"Model used: {model_name}")
    print("=" * 60)

    # Helper function to print headers and enforce rate limiting
    def call_with_delay(func, *args, **kwargs):
        time.sleep(15) # Enforce 15 seconds delay to respect 5 RPM free tier limit
        return func(*args, **kwargs)

    # 1. PARAMETER: maxOutputTokens
    print("\n[EXPERIMENT 1] Parameter: maxOutputTokens")
    print("Testing max_output_tokens = 10 (should truncate response prematurely)")
    try:
        response = call_with_delay(
            client.models.generate_content,
            model=model_name,
            contents="Explain how photosynthesis works in detail.",
            config=types.GenerateContentConfig(max_output_tokens=10)
        )
        print("Prompt: 'Explain how photosynthesis works in detail.'")
        print(f"Output (truncated):\n{response.text}")
        print(f"Finish Reason: {response.candidates[0].finish_reason if response.candidates else 'N/A'}")
    except Exception as e:
        print(f"Error in maxOutputTokens experiment: {e}")
    print("-" * 60)

    # 2. PARAMETER: temperature
    print("\n[EXPERIMENT 2] Parameter: temperature")
    print("Prompt: 'Write a one-sentence description of a futuristic city.'")

```

```

print("\n--- Low Temperature (0.0) Run 3 times (expect identical/deterministic outputs) ---")
for i in range(3):
    try:
        response = call_with_delay(
            client.models.generate_content,
            model=model_name,
            contents="Write a one-sentence description of a futuristic city.",
            config=types.GenerateContentConfig(temperature=0.0)
        )
        print(f"Run {i+1}: {response.text.strip()}")
    except Exception as e:
        print(f"Error in low temp run {i+1}: {e}")

print("\n--- High Temperature (1.0) Run 3 times (expect diverse/creative outputs) ---")
for i in range(3):
    try:
        response = call_with_delay(
            client.models.generate_content,
            model=model_name,
            contents="Write a one-sentence description of a futuristic city.",
            config=types.GenerateContentConfig(temperature=1.0)
        )
        print(f"Run {i+1}: {response.text.strip()}")
    except Exception as e:
        print(f"Error in high temp run {i+1}: {e}")
print("-" * 60)

# 3. PARAMETER: stopSequences
print("\n[EXPERIMENT 3] Parameter: stopSequences")
print("Testing stop_sequences=['process', 'chlorophyll'] (should stop when these words are generated)")
try:
    response = call_with_delay(
        client.models.generate_content,
        model=model_name,
        contents="Explain photosynthesis in a simple way.",
        config=types.GenerateContentConfig(stop_sequences=["process", "chlorophyll"])
    )
    print("Prompt: 'Explain photosynthesis in a simple way.'")
    print(f"Output (stopped early):\n{response.text}")
    print(f"Finish Reason: {response.candidates[0].finish_reason if response.candidates else 'N/A'}")
except Exception as e:
    print(f"Error in stopSequences experiment: {e}")
print("-" * 60)

# 4. PARAMETER: stream (generateContentStream)
print("\n[EXPERIMENT 4] Parameter: stream (generateContentStream)")
print("Streaming output chunk-by-chunk:")
print("Prompt: 'Write a short poem about the ocean.'")
print("Output chunks: ", end="")
try:
    time.sleep(15) # Enforce rate limit before calling stream
    response_stream = client.models.generate_content_stream(
        model=model_name,
        contents="Write a short poem about the ocean."
    )
    for chunk in response_stream:
        print(f"[{chunk.text}]", end="", flush=True)
    print()
except Exception as e:
    print(f"\nError in streaming experiment: {e}")
print("-" * 60)

# 5. PARAMETER: responseLogprobs & logprobs
print("\n[EXPERIMENT 5] Parameters: responseLogprobs & logprobs")
print(f"Attempting to query logprobs on {model_name}...")
try:
    response = call_with_delay(
        client.models.generate_content,
        model=model_name,
        contents="Hello",
        config=types.GenerateContentConfig(response_logprobs=True, logprobs=5)
    )
    print("Success! Logprobs returned:")
    print(response)
except Exception as e:
    print("Expected Error Caught!")
    print(f"Error Message: {e}")
    print(f"Explanation: Gemini Flash models do not support the logprobs feature. Pro models support it, but are restricted.")
print("-" * 60)

if __name__ == "__main__":
    run_experiments()

```